Carnegie Mellon University

Heinz College

INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

EPT: Week 5

"What a tangled web we weave"

Agenda

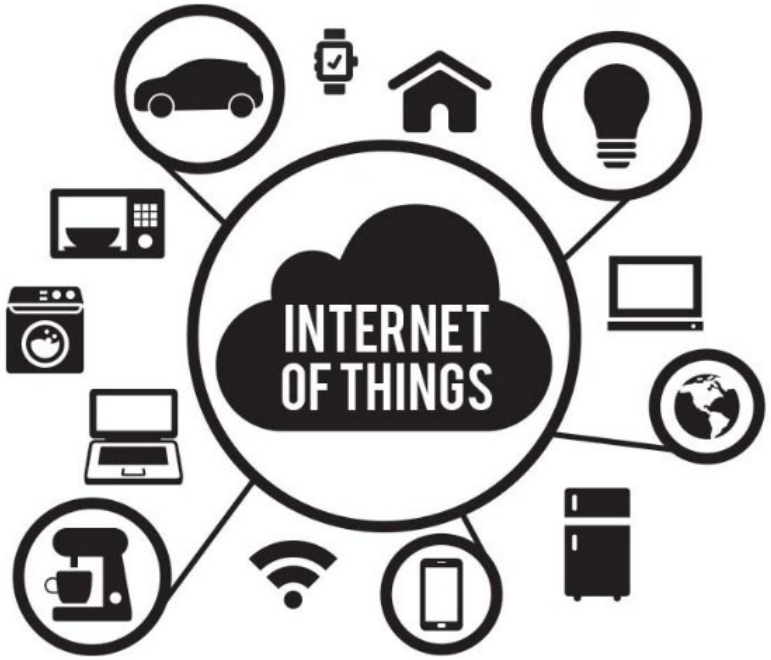
- Background
- Tools
  - Scanners
  - Content Discovery
  - Visual Inventory
- Web Vulnerabilities
  - OWASP Top Ten
- Shell Problems
  - Web shells
  - No TTY

Why web?

- Arguably the most prevalent type of vulnerability found on networks
  - The initial cause of breaches
  - Causes loss of customer data
  - DDoS
- Organizations have started to master the perimeter firewall
  - Only Web services are being exposed by most organizations
- Cloud is mostly web

Web is everywhere

- Classic uses
  - Information, user input, e-commerce, data lookup
- Infrastructure devices
  - Routers, switches, printers, network storage
- Embedded devices
  - Baseboard management controller, virtual KVM, management devices
- Internet of Things (IoT)
  - Lighting, thermostats, refrigerators, washing machines, alarm systems



Why so Vulnerable?

- Complexity causes errors
  - Production apps are big and hairy, many moving parts
- Simplicity causes oversight
  - Its just a little web app, what can go wrong?
- Code reuse
  - Many lightweight web servers used for many apps
  - Common core components/libraries/languages
    - Heartbleed, Java, PHP, JSP, AJAX, Coldfusion
- Popular sites have a target on their back
- Many instances cause lazy configurations (or none)
  - Printers, VOIP phones
- Small apps get overlooked and don't receive security review

Tools

Web Discovery / Scanners

- So you've encountered a web app, now what?
  - Browse around to understand what it is and what its supposed to do
  - Can you submit data?
  - Can you generate errors?
- Automated tools assist with:
  - Spidering the application
  - Finding hidden pages/applications
  - Checking versions, banners, headers
  - Discovering vulnerabilities
    - Passive or active
  - Uncovering restrictions

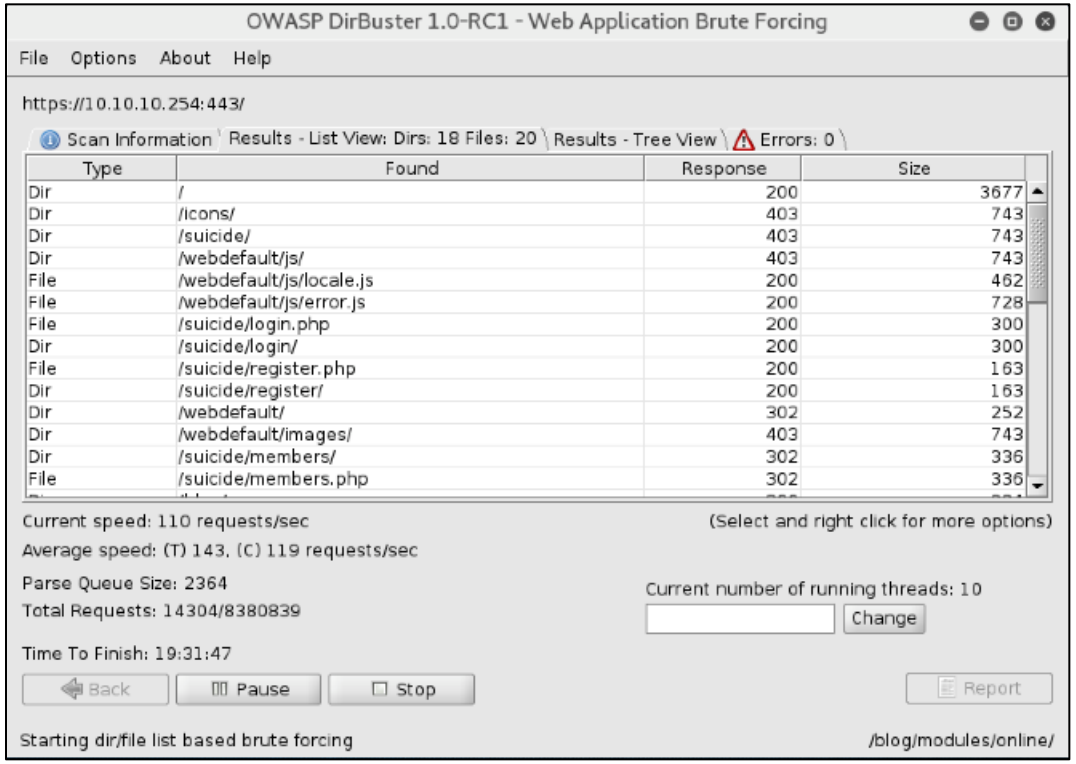
Nikto

- Lightweight, open-source web application scanner
- Checks for dangerous files, unpatched versions, server options, and misconfigurations
- Good for initial scans of small applications
- Quickly identifies vulnerabilities and hidden pages

```
root@kali:~# nikto -h 10.10.10.254
- Nikto v2.1.6
---------------------------------------------------------------------------
+ Target IP:          10.10.10.254
+ Target Hostname:    10.10.10.254
+ Target Port:        80
+ Start Time:         2016-02-15 22:16:57 (GMT-5)
---------------------------------------------------------------------------
+ Server: Apache
+ Server leaks inodes via ETags, header found with file /, fields: 0xd5e 0x4c998
45da77c0
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user a
gent to protect against some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user agent
to render the content of the site in a different fashion to the MIME type
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Uncommon header 'tcn' found, with contents: list
+ Apache mod_negotiation is enabled with MultiViews, which allows attackers to e
asily brute force file names. See http://www.wisec.it/sectou.php?id=4698ebdc59d1
5. The following alternatives for 'index' were found: index.html
+ Allowed HTTP Methods: GET, HEAD, POST, OPTIONS
+ Cookie PHPSESSID created without the httponly flag
+ OSVDB-2695: /photo/: My Photo Gallery pre 3.6 contains multiple vulnerabilitie
```

Dirbuster

- Brute-forces directories within a web root
  - Find hidden sites
- Can take custom word lists
  - /usr/share/dirbuster/wordlists
- Multi-threaded Java app
- Can take a long time



Additional Discovery Tools

- Dirb
  - Lightweight command line web content discovery tool
  - Wordlists usually in /usr/share/wordlists/dirb
- Gobuster
  - Golang = Fast
  - Can find files/directories/DNS subdomains/Virtual Hosts
- WhatWeb
  - Identifies web sites from the CMS down to the webserver version
- Word Lists
  - https://github.com/danielmiessler/SecLists

Visual Inventory Tools

- Eyewitness/GoWitness
  - Python and Golang version
  - Basic screenshotting and application discovery tools
  - Supports Nmap/Nessus/text input
- Aquatone
  - Golang
  - Pretty report
- WitnessMe
  - New in development tool
  - Shows promise

Other Scanners

- WPScan
  - Web vulnerability scanner specific to Wordpress
- Burp Suite
  - Highly effective web proxy and scanner
  - Intercept and manipulate traffic
    - MiTM attacks
  - Full-featured: Spider, Intruder, Repeater
  - Extensible via BApp Store extensions
- OWASP ZAP
  - Open Source
  - Full featured and still in development
  - Complimentary to Burp

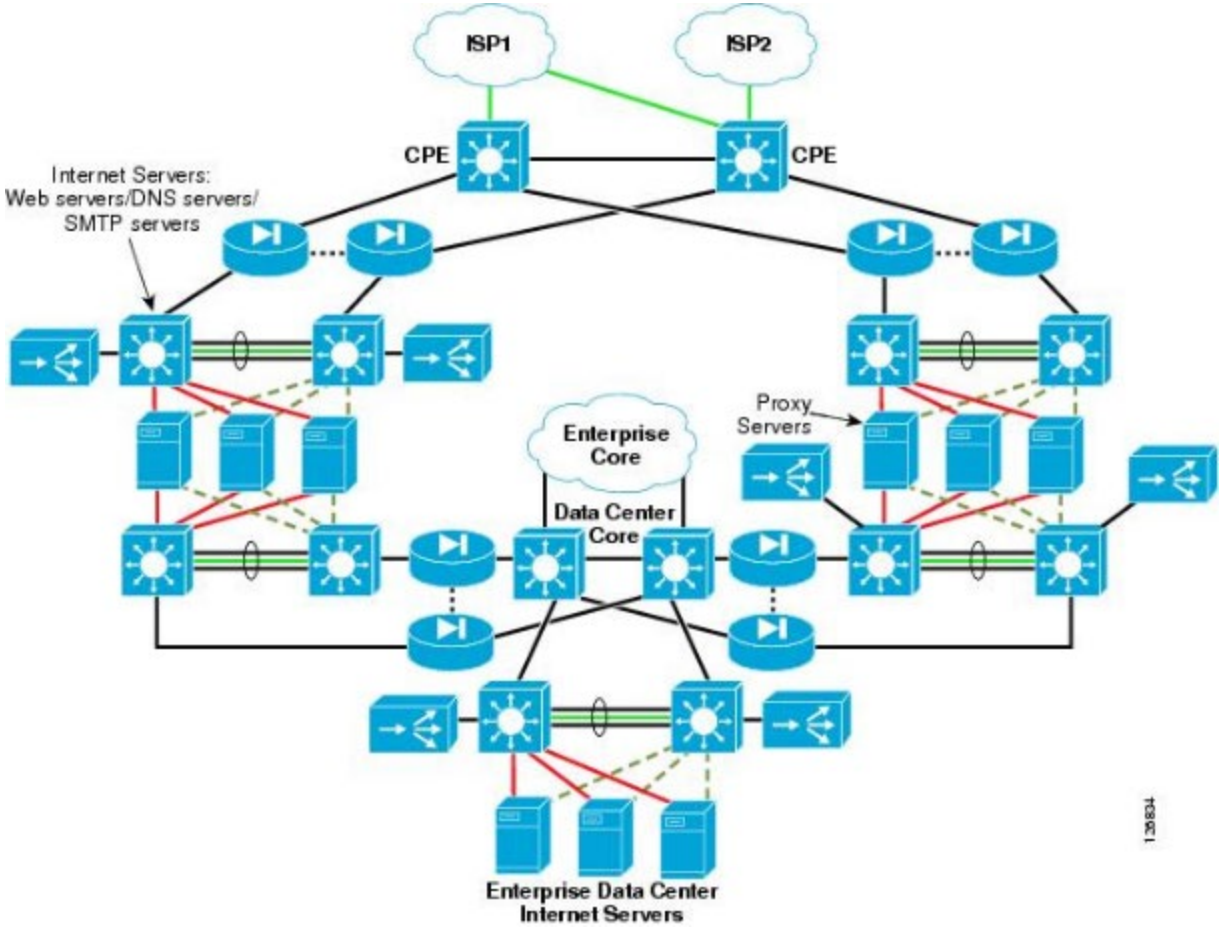
Professional Scanners

- Acunetix
  - $4.5K to $43K
  - Combines many features of open-source scanners in one package
- Netsparker
  - $5K to $30K
  - Provides POCs for identified issues
- Qualys
  - $2K to $5K
  - Offers per scan pricing

Web Vulnerabilities

Types of Web Attacks

- Web applications can be very complex and so can their exploits
  - It can be a task to get necessary access for testing
- Many variables:
  - Code base
  - Data flow
  - Intended logic
  - Connected components
  - Network layout
  - Defensive measures
    - WAF, IDS, Proxy, IPS

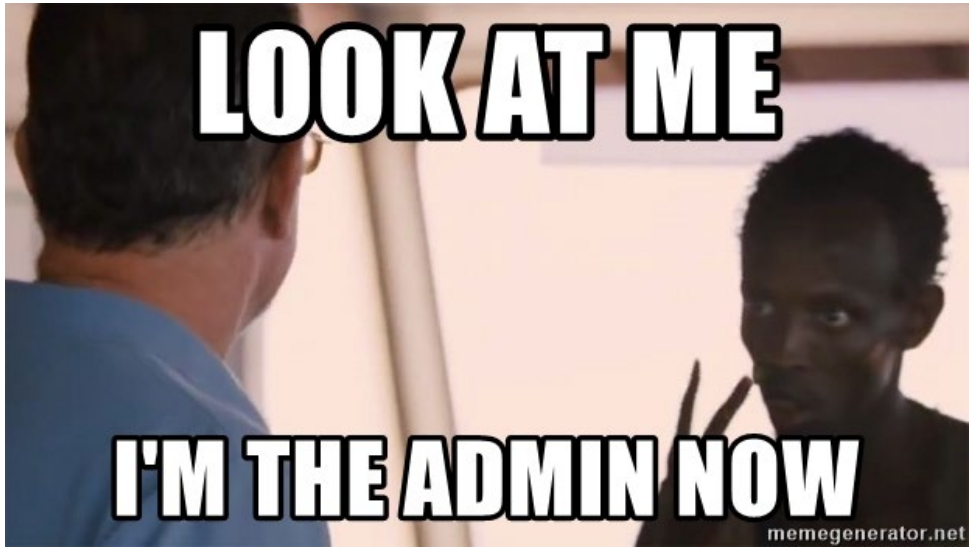


OWASP Top Ten

- Community-driven list of the ten most common web application security risks
- Published by the Open Web Application Security Project (OWASP)
  - OWASP is an open community dedicated to enabling organizations to develop, purchase, and maintain applications and APIs that can be trusted.
- https://owasp.org/Top10/

A01:2021 - Broken Access Control

- Access control defines what users are or are not allowed to do
- Permissions-Based
  - Standard User
  - Privileged User
  - Admin
- Failures lead to unauthorized disclosure, modification, or destruction of data or performing a business function outside the user's limits
- Prevention:
  - Implement access controls
  - Deny by default



Path Traversal

- Occurs when software uses unsanitized, user-supplied input to construct a path name
- Attackers can leverage special elements such as “..” and “/” to cause the path name to resolve outside of a restricted (web) directory. Can lead to disclosure of sensitive files including:
  - Configuration files
  - OS files
- Example:
  - http://example.com/get-files?file=report.pdf
  - http://example.com/get-files?file=../../../../etc/passwd

https://cwe.mitre.org/data/definitions/22.html

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Insecure Direct Object Reference (IDOR)

- Occurs when user supplied input is unvalidated and direct access to an object is provided
- Access control should be implemented ensuring users cannot act outside of their intended permissions (only accessing their own account information)
- Example:
  - https://example.com/app/accountInfo?acct=myaccount
  - https://example.com/app/accountInfo?acct=notmyacct

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Cross-Site Request Forgery (CSRF)

- Occurs when applications do not validate if a request was intentionally sent by the user who submitted the request
- Attackers can trick a user into making an unintentional request via URL, image load, etc., resulting in exposure of data or unintended code execution

- https://cwe.mitre.org/data/definitions/352.html

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Cross-Site Request Forgery (CSRF)

Form:

```
<form action="/url/profile.php" method="post">

<input type="text" name="firstname"/>
<input type="text" name="lastname"/>
<input type="text" name="email"/>
<input type="submit" name="submit"
value="Update"/>

</form>
```

Malicious Script:

```
<SCRIPT>
function SendAttack () {form.email =
"attacker@example.com";
// send to profile.php
form.submit();}
</SCRIPT>

<BODY onload="javascript:SendAttack();">

<form action="http://victim.example.com/profile.php"
id="form" method="post">
<input type="hidden" name="firstname" value="Funny">
<input type="hidden" name="lastname" value="Joke">
<input type="hidden" name="email">
</form>
```

Open Redirect

- Occurs when an application accepts a user-supplied input via a URL parameter, causing the web application to redirect the request to the specified URL
- Attackers can leverage open redirects in phishing campaigns to give a more trustworthy appearance
  - Can be chained with other vulnerabilities to bypass filters
- Example:
  - http://example.com/example.php?url=http://safe.example.com
  - http://example.com/example.php?url=http://malicious.example.com
- https://cwe.mitre.org/data/definitions/601.html

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

A02:2021 - Cryptographic Failures

- Deals with the cryptographic protection of data either in transit or at rest
- Attackers can eavesdrop network traffic if a site doesn't use or enforce TLS. If sensitive data is not encrypted when stored in a database, it risks being exposed via SQL injection or other vulnerability
- Common Vulnerabilities:
  - Use of legacy protocols
  - Lack of/weak encryption of sensitive data at rest
  - Cleartext transmission of sensitive information

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

A03:2021 - Injection

- Injection vulnerabilities occur when an attacker supplies malicious data that is interpreted by an application
  - Any source of data can be injection vectors
    - Parameters
    - Headers
    - Environment variables
- Very prevalent, particularly in legacy code bases
- Can be found via code reviews or by scanning/fuzzing inputs
- Prevention:
  - Use of secure APIs rather than interpretation
  - Parameterized queries
  - Input validation/output encoding

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Command Injection

- Occurs when an application constructs an OS command using user-supplied input, but it sanitize special elements that could modify the intended OS command
- Attackers can leverage command injection vulnerabilities to execute arbitrary commands on the host operating system
- Common operators include:
  - ;
  - &&
  - |
- https://cwe.mitre.org/data/definitions/78.html

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Command Injection

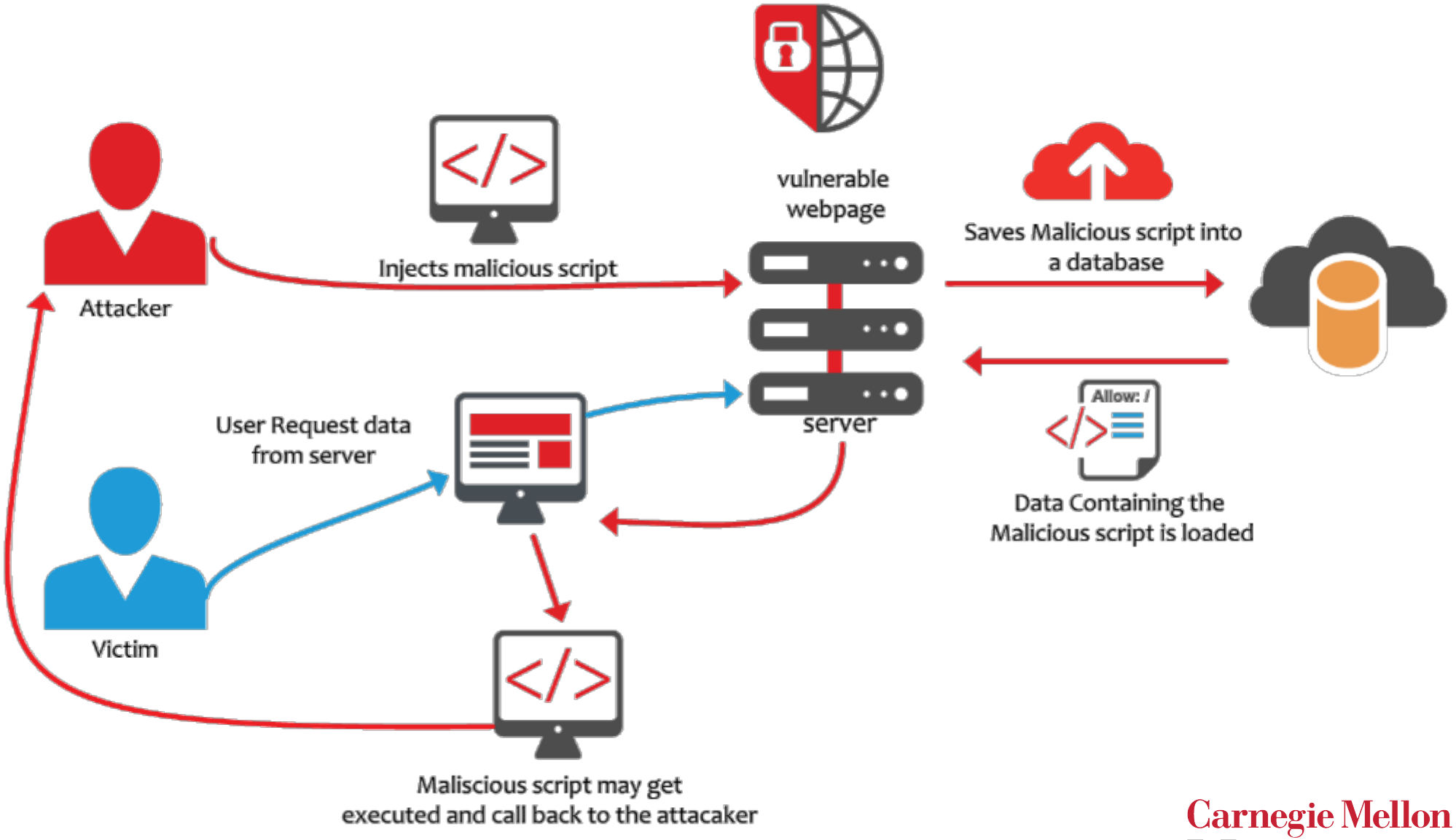
- Vulnerable Code:
  - <?php print("Please specify the name of the file to delete"); $file=$_GET['filename']; system("rm $file"); ?>
- Attacker Enters:
  - http://example.com/delete.php?filename=bob.txt;id
- Result:
  - Please specify the name of the file to delete uid=33(www-data) gid=33(www-data) groups=33(www-data)

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Cross-site Scripting (XSS)

- Method to inject malicious code into web applications
  - Client-side attack; malicious code is run by the user's browser
  - JavaScript is most popular but any browser readable code is possible (HTML, Ajax, etc.)
  - Injectable through forms, links, or other client-side attacks
  - The target is the user, web server is just facilitating
- Reflective XSS
  - Malicious code is sent to web server and then bounced back to user browser and executed
- Persistent (Stored) XSS
  - Malicious code is stored on the webserver and becomes permanent on the site
  - User browser executes upon visiting the site

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT



Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Using XSS

- XSS is often dismissed as not being worthwhile or a valuable attack vector
  - Good information can be garnered from XSS attacks
  - They do usually take multiple steps (client-side)
  - Often not used in pentests but reported on
- JavaScript is used to control the behavior of web pages.
  - Window.location: Get's current URL and redirects to a new page
  - Document.cookie: Reads the cookies of the current session
- Session IDs
  - Can be used as "authentication" to access a web app

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

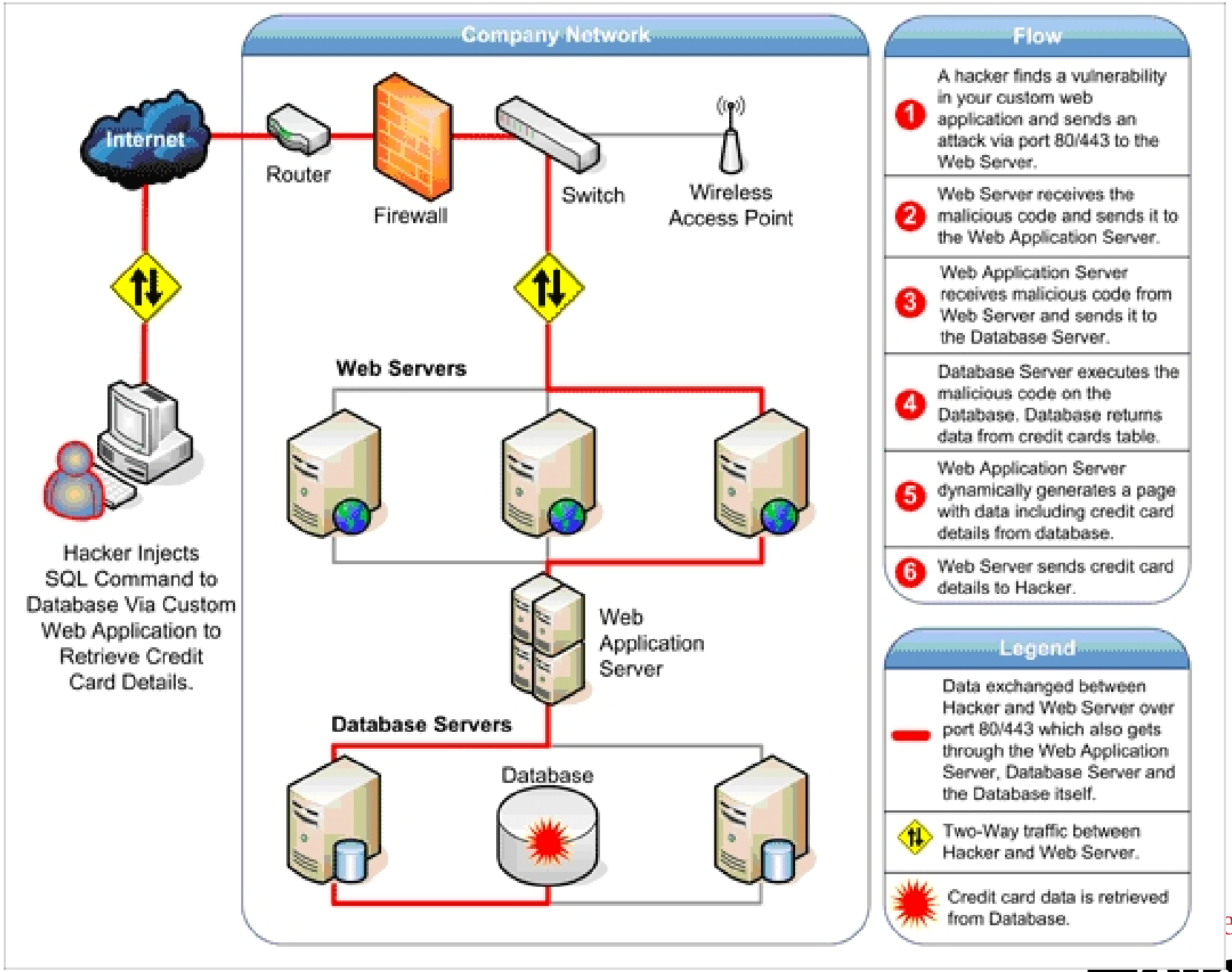
File Inclusion

- Method to inject a file onto a webserver or have the server process the file
  - Improper input validation
  - Common code, dynamic file include
  - Can be difficult to exploit, need to be precise
- Remote File Inclusion (RFI)
  - Forces the web app to process a file hosted at a remote location
  - Gives attacker ability to create and host their own payload
  - Can lead to RCE and shell access
- Local File Inclusion (LFI)
  - Restricts attack to files found local to the web server
  - Can be used to display information about the host

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

SQL Injection (SQLi)

- Method that injects SQL queries into a web application for execution on a backend database
  - Web application is the vulnerable piece (at least to start)
  - Usually caused by improperly sanitized input
  - SQL statement executed by the webserver must include user input
- Statements aim to retrieve data, modify data, insert records, or even gain a shell
  - First must enumerate the database through various techniques
- Classic SQLi: Same communication channel to execute commands and gather results
  - Error Based: Uses errors thrown by the database to enumerate the structure
  - Union Based: Combines two SELECT statements together
- Blind SQLi: Direct results of queries cannot be seen by the attacker, interpreted by the web app
  - Boolean Based: Sending TRUE/FALSE statements to the database and observing how the web app responds
  - Time Based: Sending time commands to the database and measuring when the web app responds



e Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Simple SQLi Example

- # Define POST variables
  uname = request.POST['username']
  passwd = request.POST['password']
- # SQL query vulnerable to SQLi sql = "**SELECT** credit_card **FROM** users **WHERE** username='" + ***uname*** + "' **AND password**='" + ***passwd*** + "'"
- Attacker enters:
  - uname = Bob
  - passwd = s3kr3t' OR 1=1
- **SELECT** credit_card **FROM** users **WHERE** username='Bob' **AND password**='**s3kr3t' OR** 1=1'

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

SQLMap

- Automated tool used to detect SQL injection
  - Rapidly ”brute-forces” parameters
  - Support for many different types of database
  - Includes password hash cracking and direct connections to the DB
- Success allows for enumeration of tables and dumping of data
- Mainly used to test GET parameters
  - POST supported; must provide data
- Level (1-5): Increases the number of tests to try
- Risk (1-3): Increases payloads that could lead to data manipulation

```
$ python sqlmap.py -u "http://debiandev/sqlmap/mysql/get_int.php?id=1" --batch
                                                                  {1.0.5.63#dev}
                                                                  http://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent i
s illegal. It is the end user's responsibility to obey all applicable local, state and fed
eral laws. Developers assume no liability and are not responsible for any misuse or damage
 caused by this program

[*] starting at 17:43:06

[17:43:06] [INFO] testing connection to the target URL
[17:43:06] [INFO] heuristics detected web page charset 'ascii'
[17:43:06] [INFO] testing if the target URL is stable
[17:43:07] [INFO] target URL is stable
[17:43:07] [INFO] testing if GET parameter 'id' is dynamic
[17:43:07] [INFO] confirming that GET parameter 'id' is dynamic
[17:43:07] [INFO] GET parameter 'id' is dynamic
[17:43:07] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable
 (possible DBMS: 'MySQL')
```

SQL Map Commands

- # sqlmap –u ”http://banksite/login.asp?username=value&password=value”
  - The vulnerability exists within the site code
  - The parameters are the gateway
- --cookie “name=value; name=value”
  - Essential for authenticated sessions
- --dbs: show me the databases
- -D: set a database
- --tables: show me the tables
- -T: set a table
- --dump: show me all the data

A04:2021 - Insecure Design

- Broad category encompassing many different application design weaknesses
- Insecure Design ≠ Insecure Implementation
  - Other categories focus largely on insecure implementation
  - A perfectly secure design can still have implementation defects, an insecure design cannot be fixed by perfect implementation
- Often a result of lack of business risk profiling
- Prevention:
  - Clearly defining business requirements
  - Constant evaluation of threats (threat modeling)
  - Implementation of a secure development lifecycle

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Unrestricted File Upload

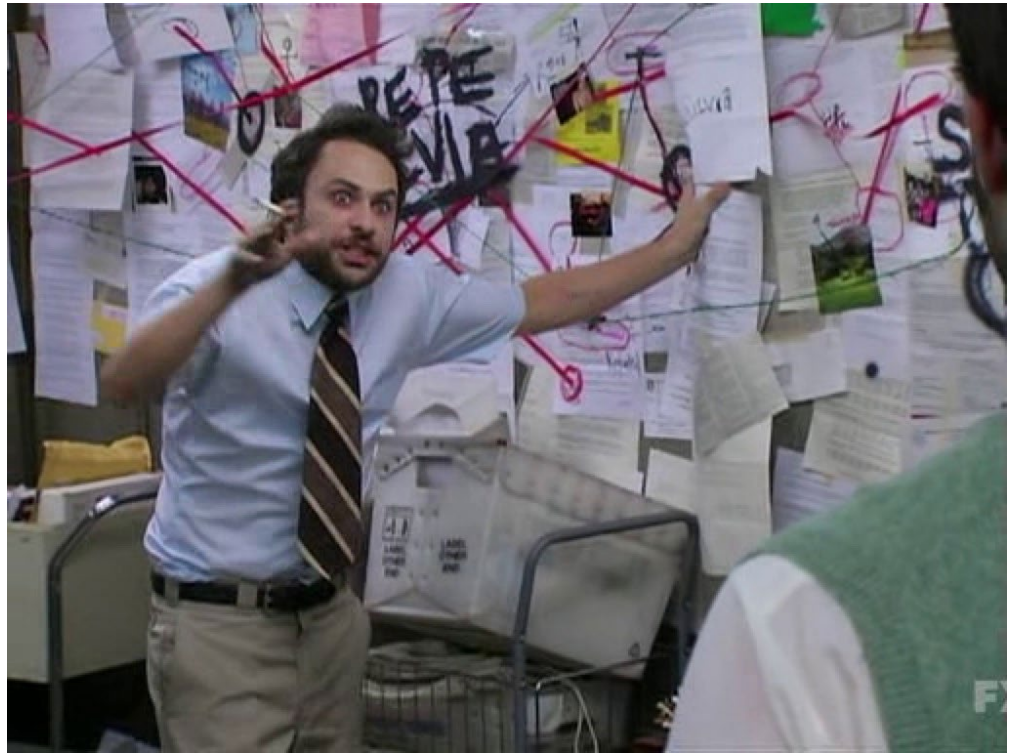
- Occurs when an application allows users to upload files of any type. These files are either automatically processed by the application or may be accessed directly by the user.
- Attacker can leverage this to upload a shell or other malicious file
  - May result in remote code execution or denial of service
- Example:
  - <form action="upload_picture.php" method="post" enctype="multipart/form-data">
    Choose a file to upload:
    <input type="file" name="filename"/>

    <input type="submit" name="submit" value="Submit"/>
    </form>

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Business Logic Errors

- Occurs when a user can manipulate the intended business logic of an application. Often involves using legitimate functionalities of the application in ways that they weren't intended to be used (or the developers didn't consider)
  - Difficult to detect automatically
- Often "non-technical" vulnerabilities
  - Instead, a result of weaknesses in business processes or application flows
- Example:
  - Making a purchase, getting reward points, then cancelling the transaction and the reward points are not returned



Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

A05:2021 - Security Misconfiguration

- Occurs when application software is not properly configured. Encompasses many different misconfigurations.
- Examples:
  - Unnecessary features enabled or installed
  - Default credentials
  - Overly information error messages/debug mode enabled
  - Disabled security features
- Prevention:
  - Separate development, QA, and production environments
  - Keep features to a minimum
  - Regularly review configurations

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

A06:2021 - Vulnerable and Outdated Components

- Occurs when software components of an application are outdated. Including:
  - Web/Application Server
  - Database Management System (DBMS)
  - Applications
  - APIs
  - Underlying operating system
- Prevention:
  - Remove unused dependencies, features, componen files, and documentation
  - Maintain an inventory of all application component
    - Frameworks, libraries, dependencies, etc.
  - Regularly update components

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

A07:2021 - Identification and Authentication Failures

- Occurs when an application fails to properly confirm a user's identity. Weaknesses may allow an attack to authenticate as another user.
- Examples:
  - Application permits brute force attacks
  - Application allows weak passwords (Ex: Password1)
  - Weak password recovery mechanism
  - Sessions not expired on logout
- Prevention:
  - Implement multi-factor authentication
  - Enforce password complexity
  - Implement a secure session manager

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

A08:2021 - Software and Data Integrity Failures

- Occurs when an application relies on plugins, libraries, or modules from untrusted sources, repositories, and content delivery networks (CDNs).

Attackers can leverage weaknesses to upload their own updates, or injection malicious code into widely-used dependencies

- Example:
  - SolarWinds Orion Attack – SolarWinds distributed a malicious update to > 18,000 organizations
- Prevention:
  - Ensure updates are coming from trusted repositories
  - Use digital signatures to verify downloads
  - Perform reviews of code/configuration changes

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

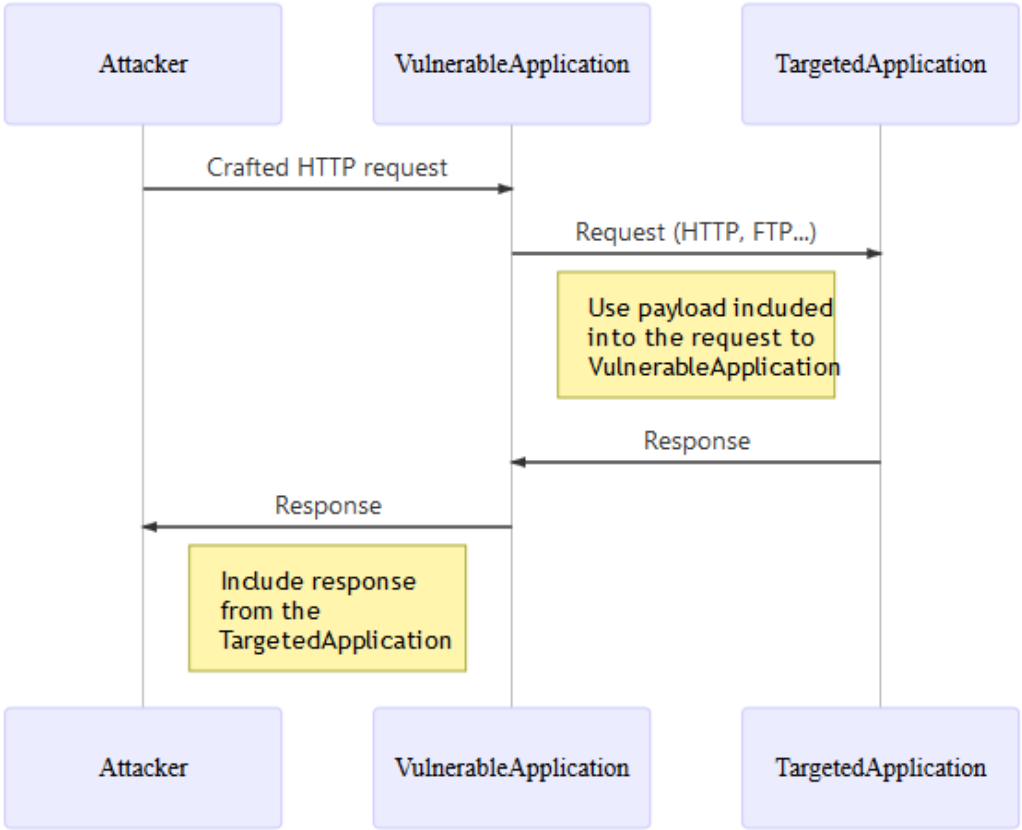
A09:2021 - Security Logging and Monitoring Failures

- Failure to detect, escalate, and respond to active breaches. Logging and monitoring is also critical for disaster recovery, including accountability, visibility, and forensics.
- Challenging to test during a penetration test, often requires interviewing the client
  - ”Did you see x?”
- Prevention:
  - Ensure login, access control, and server-side activity is logged with an appropriate level of attribution
  - Ensure logs are stored in an appropriate format and time period

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

A10:2021 - Server-Side Request Forgery (SSRF)

- Occurs when a web application makes a request without validating the user-supplied URL. Attackers can leverage an SSRF to bypass firewalls, or other network controls.
- Cloud services increase the impact of SSRF vulnerabilities
- Can leverage a variety of protocols and schemes (FTP, SMB, SMTP, Gopher, etc.)
- Prevention:
  - Sanitize user-supplied input
  - Only allow requests to URLs on an allow list
  - Do not send responses to clients



https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Shell Problems

- Web applications and browsers are very powerful but still limited when compared to a full OS environment
- Shellcode is limited to what the web application can process
  - PHP/meterpreter
  - Results in non-interactive shells
- Web Shells
  - Fully functional ”web apps” that can be uploaded and run on the web server
  - http://pentestmonkey.net/category/tools/web-shells
- No interactive shell?
  - Use Expect script
  - python -c ’import pty; pty.spawn(“/bin/sh”)’
  - /usr/bin/script -qc /bin/bash /dev/null

Resources

- http://excess-xss.com/
- http://www.acunetix.com/websitesecurity/cross-site-scripting/
- http://www.acunetix.com/websitesecurity/sql-injection/
- http://www.acunetix.com/websitesecurity/sql-injection2/
- http://www.w3schools.com/sql/sql_injection.asp
- http://www.veracode.com/security/sql-injection

Assignment: PWN Challenge #4

- The objective of this challenge is to utilize the tools and techniques you have been taught to discover and exploit what you can within the authorized scope.
- To get credit for the assignment you will be required to create an assessment report.
  - Should contain your findings along with their associated impact.
  - Should create context for your customer (your instructors) so that informed decisions can be made to fix their network.
  - Should contain the proof hash values and include enough detail that your results could be reproduced by your instructors.
- This assignment builds on the previous pwn challenge.
- The assignment deliverable is a report. See Canvas for more detailed rubric.
- Note: Vulnerabilities/findings may not show up in vulnerability scans.
- 1 Proof in PWN4

Bonus Assignment: PWN Challenge #5

- Write up only needs basic screenshots of your attack path and simple explanation.
- Worth up to 25 pts.
- 15 pts for local.txt
- 10 pts for proof.txt
- No assistance will be provided!

Labs

- STEPfwd Labs
  - Exploiting Web Application
- TryHackMe Labs
  - https://tryhackme.com/room/webfundamentals
  - https://tryhackme.com/room/rpwebscanning
  - https://tryhackme.com/room/sqlilab
  - https://tryhackme.com/room/injection
  - https://tryhackme.com/room/lfibasics
  - https://tryhackme.com/room/ffuf
  - https://tryhackme.com/room/learnssti
  - https://tryhackme.com/room/owasptop10
  - https://tryhackme.com/room/learnowaspzap
  - https://tryhackme.com/room/burpsuitebasics
  - https://tryhackme.com/room/burpsuiterepeater
  - https://tryhackme.com/room/owaspjuiceshop
  - https://tryhackme.com/room/owaspmutillidae
  - https://tryhackme.com/room/webgoat
- Other Labs
  - https://portswigger.net/web-security